
LLM Chat App — Project Report

1. API Structure

The API is built with FastAPI and follows REST conventions. All protected routes require a JWT access token in the `Authorization: Bearer <token>` header.

Auth Routes

Method	Endpoint	Auth Required	Description
POST	/auth/register	No	Register with username, email, password
POST	/auth/login	No	Login, returns access token + sets refresh cookie
POST	/auth/refresh	No	Uses refresh token cookie to issue new access token
POST	/auth/logout	No	Deletes refresh token from Redis
GET	/auth/github	No	Redirects to GitHub OAuth page
GET	/auth/github/callback	No	GitHub redirects here with code, issues tokens

Chat Routes

Method	Endpoint	Auth Required	Description
POST	/chats/	Yes	Create a new chat thread
GET	/chats/	Yes	Get all chats for current user
GET	/chats/{id}	Yes	Get a single chat with all its messages
PATCH	/chats/{id}	Yes	Update chat title
DELETE	/chats/{id}	Yes	Delete chat and all its messages

Message Routes

Method	Endpoint	Auth Required	Description
GET	/chats/{id}/messages	Yes	Get all messages in a chat
POST	/chats/{id}/messages	Yes	Send message, get full LLM response
POST	/chats/{id}/messages/stream	Yes	Send message, get streaming LLM response

How Clients Use the API

1. Client calls POST /auth/login → receives access_token in response body
 2. Client stores access_token in localStorage
 3. Every subsequent request includes header: Authorization: Bearer <token>
 4. When access token expires (401 response) → client calls POST /auth/refresh → gets new access token
 5. For streaming responses, client uses the Fetch API with a ReadableStream reader to read Server-Sent Events token by token
-

2. Code Structure

```
llm-chat-app/
├── app/
│   └── main.py                # App entry point, middleware, router
├── registration
│   ├── core/
│   │   ├── __init__.py       # Loads all env variables via python-dotenv
│   │   ├── database.py       # MongoDB connection (motor async client)
│   │   ├── redis.py          # Redis connection (redis.asyncio client)
│   │   └── dependencies.py    # get_current_user - JWT auth middleware
│   └── models/
│       └── user.py           # UserInDB, RegisterRequest, LoginRequest,
│                               UserResponse
│   └── chat.py               # Chat, Message, CreateChatRequest,
│                               SendMessageRequest
│   └── controllers/
│       ├── auth_controller.py # /auth routes - register, login, refresh,
│       │                       logout, github
│       └── chat_controller.py # /chats routes - CRUD + message sending +
│                               streaming
│   └── services/
│       └── auth_service.py    # Password hashing, JWT creation, refresh
├── token logic
│   ├── chat_service.py       # Chat/message CRUD + Redis caching logic
│   ├── github_service.py     # GitHub OAuth token exchange + user fetch
│   └── llm_service.py        # Mistral AI API calls (normal + streaming)
├── frontend/
│   └── src/
│       ├── api.js            # Axios instance with auth interceptors
│       ├── context/
│       │   └── AuthContext.jsx # Global auth state, login/logout functions
│       ├── pages/
│       │   ├── Login.jsx     # Login form + GitHub OAuth button
│       │   ├── Register.jsx  # Registration form
│       │   └── Chat.jsx       # Main chat page with streaming
│       └── components/
│           ├── Sidebar.jsx    # Chat list, new chat button, user info
│           └── MessageBubble.jsx # Individual message display
├── .env                      # Environment variables (not committed to git)
└── requirements.txt          # Python dependencies
```

└─ README.md

Setup and run instructions

Key Modules Explained

app/main.py — Creates the FastAPI app, registers CORS middleware, registers all routers, and manages startup/shutdown lifecycle using the modern `lifespan` context manager.

app/core/dependencies.py — The `get_current_user` function acts as auth middleware. Any route that includes `Depends(get_current_user)` is automatically protected. It extracts the Bearer token, decodes the JWT, and fetches the user from MongoDB.

app/services/auth_service.py — Contains all auth logic: bcrypt password hashing/verification, JWT access token creation/decoding, and Redis-based refresh token management (create, verify, delete).

app/services/chat_service.py — Handles all chat and message operations. Includes a Redis caching layer: messages are cached for 30 minutes on first load, cache is invalidated when new messages are added or chat is deleted.

app/services/llm_service.py — Two functions: `get_llm_response()` for standard responses and `stream_llm_response()` which is an async generator that yields tokens one by one using Mistral's streaming API.

frontend/src/api.js — Axios instance with two interceptors: one that automatically attaches the JWT token to every request, and one that automatically refreshes the token on 401 responses.

3. UI Screenshots

Login Page

localhost:5173/login

DE

All Bookmarks



Welcome back

Sign in to your account

Email

you@example.com

Password

***** 

Sign in

or continue with

 Continue with GitHub

Don't have an account? [Sign up](#)

Register Page

localhost:5173/register

All Bookmarks



Create account

Start chatting with AI today

Username

johndoe

Email

you@example.com

Password

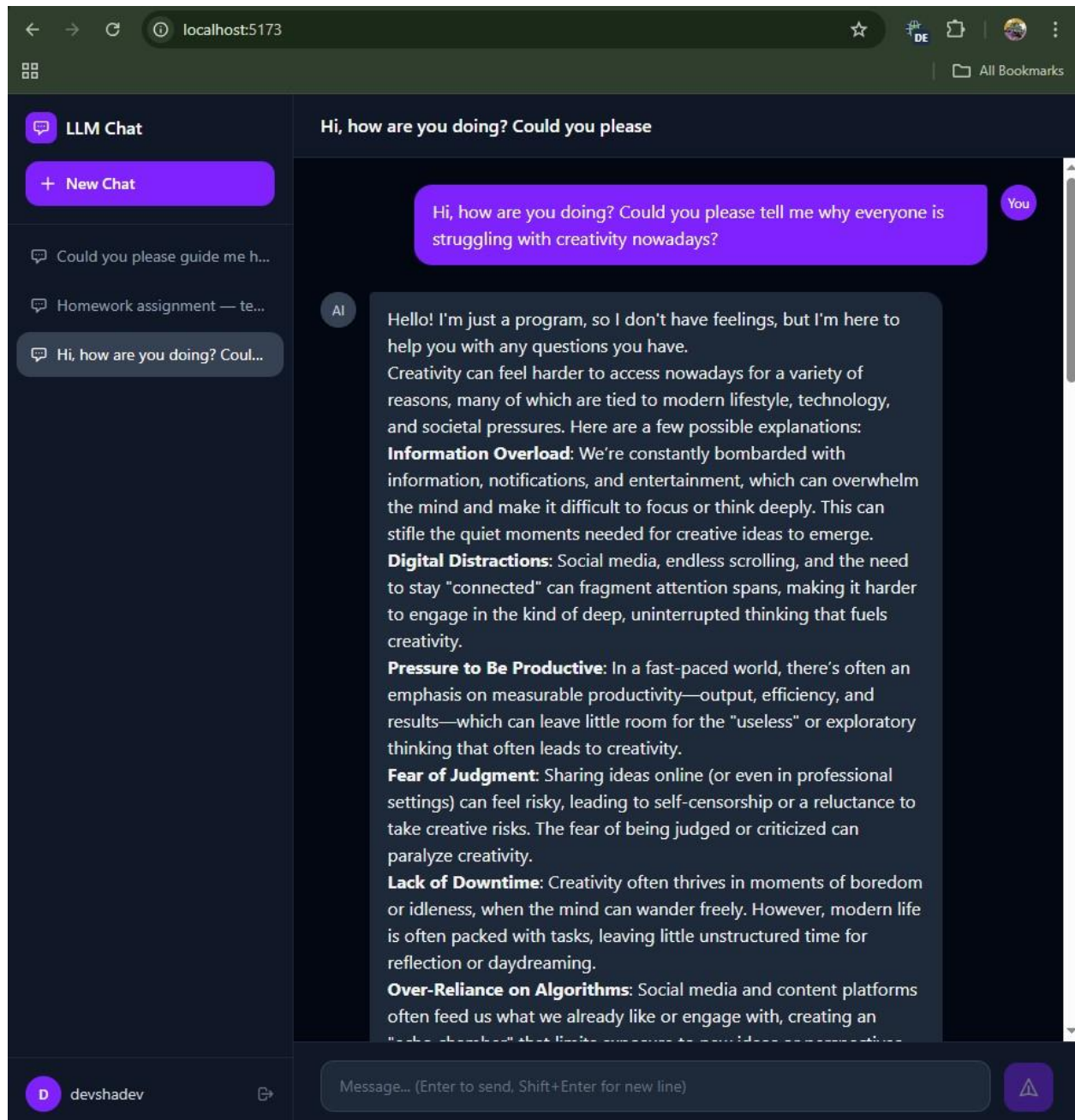
Create account

or continue with

 Continue with GitHub

Already have an account? [Sign in](#)

Chat Page:



MongoDB Tables

cloud.mongodb.com/v2/67e412ad37b8f5cd2dafef?explorer/697418392acbd242b507a2e4/llm_chat

syed umair's Org - 20...ecommerce backend

syed umair's Org - 20...ecommerce backend

My QueriesData Modelling

CLUSTERS (1)

Search clusters

Cluster0

admin

llm_chat

chats

messages

users

local

oplog.rs

sample_mflix

test

llm_chat

Cluster0 > llm_chat

View monitoringVisualize your dataCreate collectionRefresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
chats	-	36.86 kB	917.00 B	6	152.00 B	1	36.86 kB
messages	-	66.02 kB	48.70 kB	36	1.35 kB	1	36.86 kB
users	-	36.86 kB	824.00 B	3	174.00 B	1	36.86 kB

System Status: All Good
©2024 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Schema (Chats)

MongoDB Compass - cluster0.wmeay39.mongodb.net/llm_chat.chats

Connections Edit View Collection Help

Compass

My Queries

Data Modeling

CONNECTIONS (3)

Search connections

- cluster0.2hxtjif.mongodb.net
- cluster0.wmeay39.mongodb.net
 - admin
 - config
 - llm_chat
 - chats**
 - messages
 - users
- local
- sample_mflix
- test

cluster0.wmeay39.mongodb.net

cluster0.wmeay39.mongodb.net > llm_chat > chats

Documents 6 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

```
{
  "_id": ObjectId('69e6c2da1cbb8b7c7da9a593'),
  "user_id": "69e6ade2831669b7326a5e53",
  "title": "This is my first chat testing it with proud",
  "created_at": 2026-04-21T00:20:42.238+00:00,
  "updated_at": 2026-04-21T12:30:49.606+00:00
}
```

```
{
  "_id": ObjectId('69e780b7c46cc5518cd6caf1'),
  "user_id": "69e780b7c46cc5518cd6caf0",
  "title": "Why is everyone struggling with creativi",
  "created_at": 2026-04-21T13:50:55.347+00:00,
  "updated_at": 2026-04-21T13:51:47.206+00:00
}
```

```
{
  "_id": ObjectId('69ebaa8ac46cc5518cd6cafb'),
  "user_id": "69e780b7c46cc5518cd6caf0",
  "title": "Hey, I am having headache due to stomach",
  "created_at": 2026-04-24T17:38:18.027+00:00,
  "updated_at": 2026-04-24T17:44:26.195+00:00
}
```

```
{
  "_id": ObjectId('69ef4f5c3cb734562960514b'),
  "user_id": "69e6c011be4e213b59827087",
  "title": "Hi, how are you doing? Could you please ",
  "created_at": 2026-04-27T11:58:20.721+00:00,
  "updated_at": 2026-04-27T12:04:06.879+00:00
}
```

```
{
  "_id": ObjectId('69ef520a7cb2974be6df6b35'),
  "user_id": "69e6c011be4e213b59827087",
  "title": "Homework assignment - technical specific",
  "created_at": 2026-04-27T12:09:46.821+00:00,
  "updated_at": 2026-04-27T22:52:30.610+00:00
}
```

```
{
  "_id": ObjectId('69ef52797cb2974be6df6b38'),
  "user_id": "69e6c011be4e213b59827087",
  "title": "Could you please guide me how to achieve",
  "created_at": 2026-04-27T12:11:37.334+00:00,
  "updated_at": 2026-04-27T12:12:11.434+00:00
}
```

- Login page
- Register page
- Welcome / empty state screen
- Active chat with messages
- Sidebar showing multiple chats
- MongoDB Atlas collections view

4. Database Structure

The database is **MongoDB Atlas** (free M0 cluster). Database name: `llm_chat`. It contains 3 collections.

Users Collection

Stores all registered users whether they signed up with email/password or GitHub OAuth.

```
{
  "_id":           ObjectId      - auto-generated unique ID
  "username":      String        - unique display name
  "email":         String        - unique email address
  "hashed_password": String | null - bcrypt hash, null for GitHub-only users
  "github_id":     String | null  - GitHub user ID, null for email users
  "created_at":    DateTime      - registration timestamp
}
```

Chats Collection

Each document represents one conversation thread belonging to a user.

```
{
  "_id":           ObjectId      - auto-generated unique ID
  "user_id":       String        - references Users._id (ownership check on every
query)
  "title":         String        - auto-set from first message (first 40 chars)
  "created_at":    DateTime      - when chat was created
  "updated_at":    DateTime      - updated on every new message (used for sorting)
}
```

Messages Collection

Each document is one message inside a chat. Role is either "user" or "assistant".

```
{
  "_id":           ObjectId      - auto-generated unique ID
  "chat_id":       String        - references Chats._id
  "role":          String        - "user" or "assistant"
  "content":       String        - the message text
  "created_at":    DateTime      - message timestamp (used for ordering)
}
```

Redis Key Structure

Redis is used for two purposes:

Key Pattern	Value	TTL	Purpose
refresh:<token>	user_id	30 days	Refresh token session storage
chat_messages:<chat_id>	JSON array	30 minutes	Chat history cache

5. Architecture & Design Decisions

Why MCS over MVC? Since the frontend is a React SPA, the backend serves only JSON — no HTML templates. MCS (Model-Controller-Service) is the appropriate pattern: controllers handle HTTP concerns, services contain business logic, and models define data shapes.

Why MongoDB? Chat messages and user data are document-shaped and schema-flexible. MongoDB's document model maps naturally to our data without needing joins or migrations.

Why Redis for both sessions and caching? Redis excels at two things we need: fast key-value TTL storage for refresh tokens (security), and low-latency cache for frequently read chat history (performance). Using one tool for both keeps the stack simple.

JWT Flow: Access tokens are short-lived (30 min) and stored in `localStorage` for easy use in API calls. Refresh tokens are long-lived (30 days), stored in Redis server-side, and sent only via HTTP-only cookies — making them inaccessible to JavaScript and protected against XSS attacks.

Streaming: The `/messages/stream` endpoint uses FastAPI's `StreamingResponse` with Server-Sent Events (SSE) format. The frontend uses the Fetch API's `ReadableStream` to read tokens as they arrive, updating the UI in real time — giving users the familiar ChatGPT-like typing experience.
